16 SEPTEMBER 2026

HOTEL MANAGEMENT

DATABASE MANAGEMENT SYSTEM

CE3109 • MySQL Project Presentation

Prepared by

Watcharaphon Saeyong • ID 6410776

Syed Jibrael Bagheri • ID 6520031



Assumption University • Faculty of Engineering • Section 641

Presentation roadmap

01
Project idea
What problem the database solves

02
System design
ER diagram, entities and relationships

03
Implementation
Tables, keys and sample data

04
SQL demonstrations
Queries and results

05
Reflection
Challenges and future improvements

The main idea

Bring hotel operations into one connected relational database — instead of scattered files and spreadsheets.

12 entities • MySQL • 8 sample queries

Why build a hotel database?

The problem

A hotel handles guests, reservations, rooms, payments, staff and parking at the same time. If these records are disconnected, it becomes harder to find accurate information quickly.

Our solution

A relational DBMS in MySQL stores related information in separate tables, then connects them with primary keys and foreign keys.

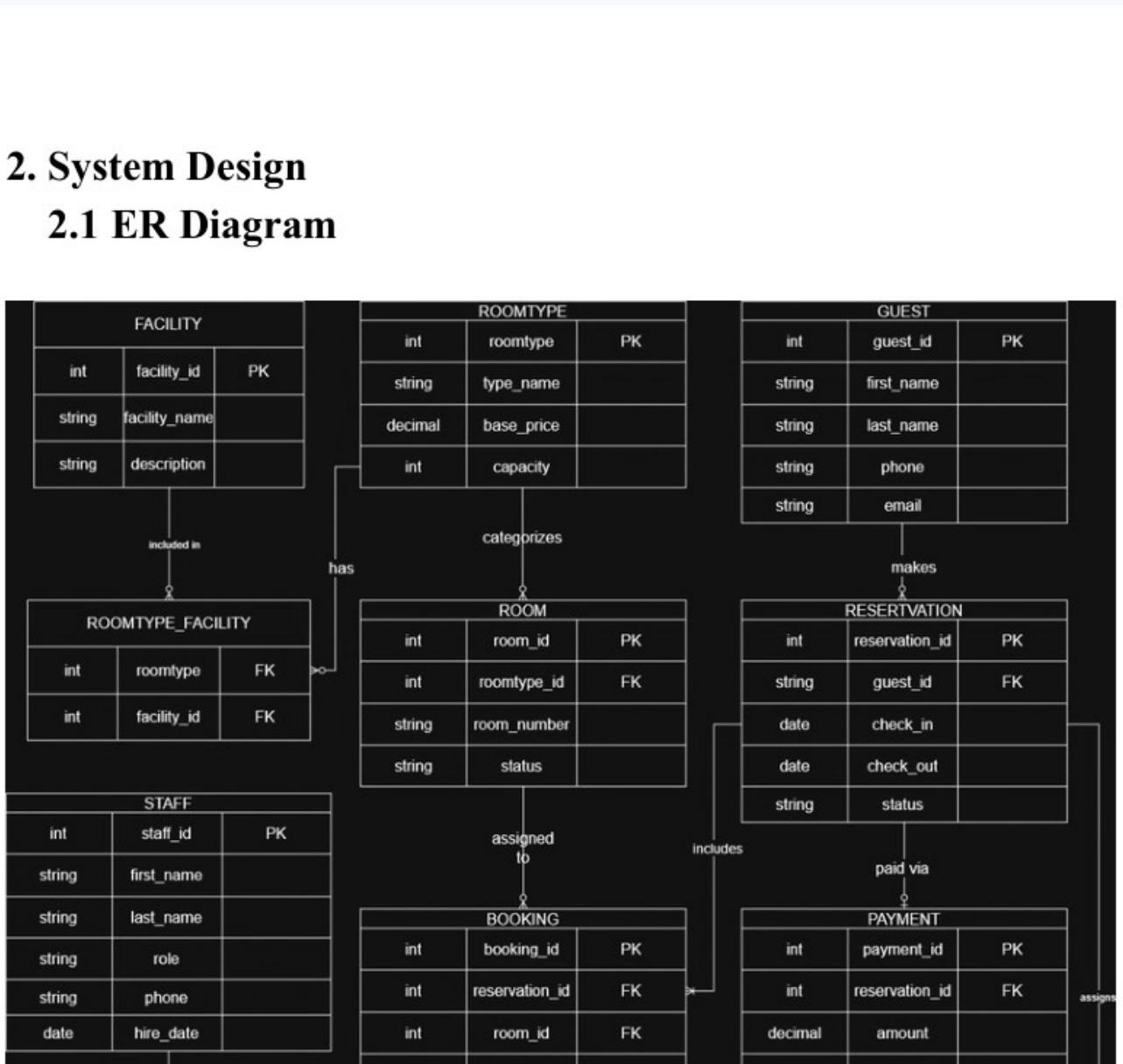
Project objectives

- Manage guest details and reservation status
- Track room types, prices, capacity and room status
- Record facilities, payments, staff salaries and parking

Scope boundaries

- Included: reservations, rooms, facilities, staff, payments and parking
- Not included: housekeeping schedules, inventory/supply management and online booking integration

System design: ER diagram



12 entities

The model separates each major hotel operation into its own table.

Key idea

Junction tables solve many-to-many relationships.

Design focus

Keep data organized and reduce repeated information.

The database has 12 entities

GUEST
Guest contact details

RESERVATION
Dates and status

BOOKING
Reservation ↔️ room link

ROOM
Room number and status

ROOMTYPE
Category, price, capacity

FACILITY
Wi-Fi, pool, spa, etc.

ROOMTYPE_FACILITY
Room type ↔️ facility

PAYMENT
Guest payment records

STAFF
Employee information

STAFF_PAYMENT
Salary/payment history

PARKING_SLOT
Parking inventory

RESERVATION_PARKING
Reservation ↔️ parking

Relationships and cardinality

GUEST → RESERVATION

1-to-many

One guest can make many reservations.

RESERVATION → BOOKING

1-to-many

One reservation may contain several rooms.

ROOMTYPE → ROOM

1-to-many

One room type can categorize many rooms.

ROOMTYPE ↔ FACILITY

many-to-many

Resolved by ROOMTYPE_FACILITY.

RESERVATION ↔ PARKING_SLOT

many-to-many

Resolved by RESERVATION_PARKING.

STAFF → STAFF_PAYMENT

1-to-many

One staff member can have many payment records.

Implementation: tables and keys

Example table structure

Column	Type	Key
guest_id	INT	**PK**
first_name	VARCHAR(50)	—
last_name	VARCHAR(50)	—
phone	VARCHAR(20)	—
email	VARCHAR(100)	—

Primary key (PK)

Uniquely identifies each row. Example: guest_id.

Foreign key (FK)

Connects a row to a record in another table.

Creation order matters

Independent tables are created first. Dependent tables come later so foreign key references are valid.

Example dependency chain



Sample data loaded into MySQL

5 **Guests**
from INSERT statements

4 **Room types**
from INSERT statements

7 **Rooms**
from INSERT statements

5 **Reservations**
from INSERT statements

5 **Staff members**
from INSERT statements

8 **Parking slots**
from INSERT statements

Example room pricing

Single = 1,500 THB • Double = 2,500 THB • Deluxe = 3,500 THB • Suite = 5,000 THB

SQL implementation: creating tables

Example: ROOMTYPE and ROOM

```
CREATE TABLE ROOMTYPE (
    roomtype_id INT PRIMARY KEY AUTO_INCREMENT,
    type_name VARCHAR(50) NOT NULL,
    base_price DECIMAL(10,2) NOT NULL,
    capacity INT NOT NULL
);

CREATE TABLE ROOM (
    room_id INT PRIMARY KEY AUTO_INCREMENT,
    roomtype_id INT NOT NULL,
    room_number VARCHAR(10) NOT NULL,
    status VARCHAR(20) NOT NULL,
    FOREIGN KEY (roomtype_id)
        REFERENCES ROOMTYPE(roomtype_id)
);
```

Why this works

ROOM references ROOMTYPE, so every room belongs to a valid room category.

Constraints used

PRIMARY KEY • FOREIGN KEY • NOT NULL • AUTO_INCREMENT

Design benefit

The database prevents invalid references and keeps related data consistent.

SQL demonstration: finding rooms and history

Query 1 — available rooms

```
SELECT r.room_id, r.room_number,
       rt.type_name, rt.base_price
FROM ROOM r
JOIN ROOMTYPE rt
  ON r.roomtype_id = rt.roomtype_id
WHERE r.status = 'Available';
```

Result from the report

101	Single	1,500
201	Double	2,500
301	Deluxe	3,500
401	Suite	5,000

Query 2 — guest reservation history

```
SELECT g.first_name, g.last_name,
       res.check_in, res.check_out,
       res.status, rm.room_number,
       b.rate_applied
FROM GUEST g
JOIN RESERVATION res
  ON g.guest_id = res.guest_id
JOIN BOOKING b
  ON res.reservation_id = b.reservation_id
JOIN ROOM rm
  ON b.room_id = rm.room_id
ORDER BY g.guest_id;
```

What this demonstrates

Multiple JOIN operations combine guest, reservation, booking and room information.

SQL demonstration: totals and grouped data

Query 3 — total revenue

```
SELECT SUM(amount) AS total_revenue
FROM PAYMENT;
```

20,000 THB

Query 4 — facilities by room type

```
SELECT rt.type_name, f.facility_name
FROM ROOMTYPE rt
JOIN ROOMTYPE_FACILITY rf ...
```

Uses a junction table

Query 5 — staff payments by role

```
SELECT s.role, SUM(sp.amount) AS
total_paid
FROM STAFF s
JOIN STAFF_PAYMENT sp ...
```

Manager: 45,000 THB
Receptionist: 56,000 THB

These queries show aggregation (SUM), JOINs and grouping (GROUP BY) — not only simple SELECT statements.

SQL demonstration: operational checks

6

Pending / cancelled reservations

Find reservations needing attention.

```
WHERE res.status IN ('Pending', 'Cancelled')
```

7

Number of rooms per type

Summarize room inventory.

```
COUNT(r.room_id) ... GROUP BY rt.type_name
```

8

Available parking slots

Avoid assigning occupied slots.

```
WHERE ps.status = 'Available' ...
```

Practical value

The database c

Challenges and future improvements

Challenge 1 — foreign key order

Tables with dependencies must be created after the tables they reference. We handled this by creating independent tables first.

Future improvement 1

Add booking constraints to prevent overlapping room bookings.

Challenge 2 — parking design

A single parking_slot_id could not support multiple cars. We introduced RESERVATION_PARKING with a fee field.

Future improvement 2

Automate parking status updates and add housekeeping and inventory modules.

The project is a working foundation that can be extended into a larger hotel information system.

Conclusion

We designed and implemented a relational hotel database in MySQL.

01

Organized data

12 connected entities cover core hotel operations.

02

SQL skills

CREATE TABLE, INSERT, JOIN, SUM, COUNT and GROUP BY.

03

Real-world thinking

Foreign keys, parking allocation and future constraints.

Thank you

Questions?